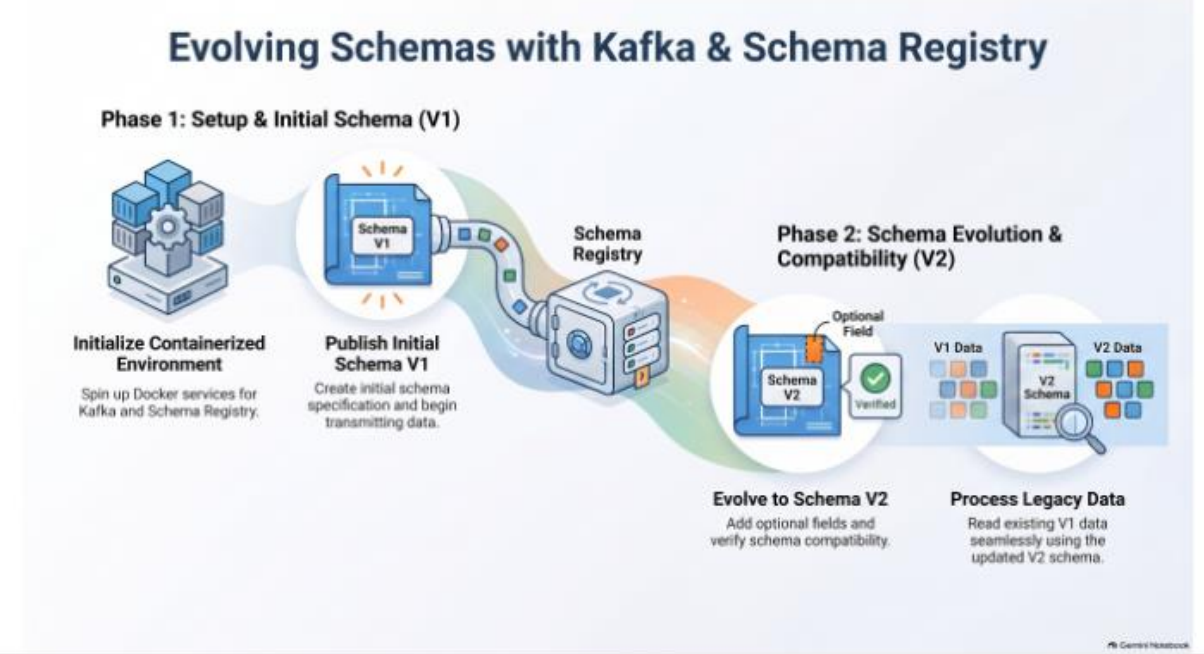
Practical 4

4	**Schema Evolution:** Use Kafka Schema Registry to demonstrate a "Backward Compatible" schema change.



In **Kafka Schema Registry**, **backward compatibility** means:

A new schema can be used to read data that was produced using the older schema.

Hospital Patient System

|

| Patient data

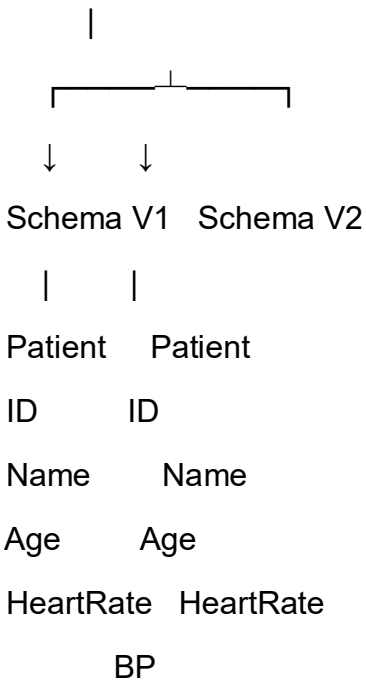
↓

Kafka

|

↓

Schema Registry



We can use **Patient Records / Hospital Monitoring Data**.

Real-world scenario

Suppose a hospital initially sends patient information to Kafka like this:

Version 1

Patient ID

Patient Name

Age

Heart Rate

Later, the hospital wants to add:

Version 2

Patient ID

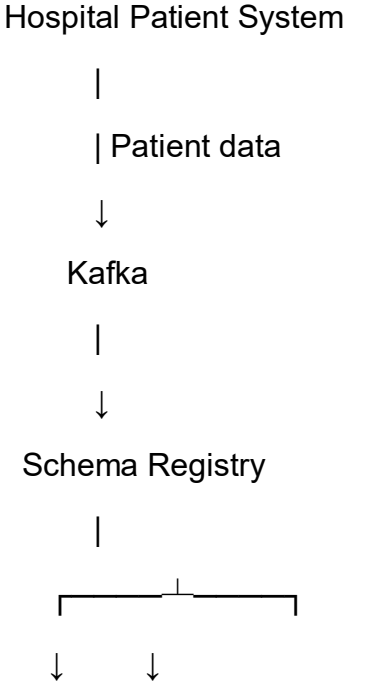
Patient Name

Age

Heart Rate

Blood Pressure

The flow will be:



What we will actually demonstrate

V1 — Original patient schema

```
{
  "patient_id": 101,
  "patient_name": "Amit",
  "age": 45,
```

```
  "heart_rate": 78
}
```

Then the hospital changes its system.

V2 — New patient schema

```
{
  "patient_id": 101,
  "patient_name": "Amit",
  "age": 45,
  "heart_rate": 78,
  "blood_pressure": "120/80"
}
```

The important part is:

```
{
  "name": "blood_pressure",
  "type": "string",
  "default": "Unknown"
}
```

Practical sequence

1. **Create hospital Kafka environment using Docker**
2. Start Kafka
3. Start Schema Registry
4. Create patient-v1.avsc
5. Register V1
6. Set compatibility to BACKWARD
7. Create patient-v2.avsc
8. Add blood_pressure
9. Register V2
10. Verify that V2 is accepted
11. Produce patient data using the new schema

12. Create an incompatible V3
13. Try registering V3
14. Observe Schema Registry rejecting V3
15. Verify all schema versions